

Hashing

Muhammad Abdulkariim

What is Hashing?

- Definition:
 - Hashing is a technique to map data of arbitrary size to fixed-size values.
 - Used for fast data retrieval, storage, and integrity checks.
- Key Components:
 - Hash Function: A function that computes the hash value.
 - Hash Table: A data structure that stores key-value pairs.

Hash Functions

- Purpose:
 - Converts input (key) into a fixed-size hash value (index).
- Properties of a Good Hash Function:
 - Deterministic: Same input always produces the same hash.
 - Uniform Distribution: Evenly distributes keys across the hash table.
 - Efficient: Computes hash value quickly.
 - Pre-image Resistance: Hard to reverse-engineer input from hash.
- Example:
 - Simple hash function: $h(\text{key}) = \text{key} \% \text{table_size}$

Hash Tables

- Structure:
 - Array of buckets/slots where data is stored.
 - Each slot is indexed by a hash value.
- Operations:
 - Insert: Compute hash, store value at the index.
 - Search: Compute hash, retrieve value from the index.
 - Delete: Compute hash, remove value from the index.

Hash Table Simulation

Calculating hash for key "apple": $97 + 112 + 112 + 108 + 101 \% 10 = 0$

apple: 5					

Collision in Hashing

- What is a Collision?
 - Occurs when two different keys produce the same hash value.
- Why Collisions Happen:
 - Hash functions map infinite inputs to finite outputs.
- Example:
 - Keys "apple" and "orange" hash to the same index.

Collision Handling

- Separate Chaining:
 - Each bucket contains a linked list of key-value pairs.
 - Colliding keys are stored in the same bucket as a list.
- Open Addressing:
 - Find another slot within the hash table.
 - Techniques:
 - Linear Probing: Check the next slot.
 - Quadratic Probing: Check slots in quadratic increments.
 - Double Hashing: Use a second hash function to find the next slot.

Applications of Hashing

- Data Retrieval:
 - Hash tables for fast lookups (e.g., dictionaries, databases).
- Cryptography:
 - Hash functions for data integrity (e.g., SHA-256, MD5).
- Caching:
 - Hash maps for quick access to cached data.
- Compilers:
 - Symbol tables for variable and function names.

Advantages

- Fast Operations:
 - Average-case time complexity: $O(1)$ for insert, search, delete.
- Space Efficiency:
 - Only stores necessary data.
- Versatility:
 - Used in databases, cryptography, caching, etc.

Challenges

- Collisions:
 - Can degrade performance if not handled properly.
- Hash Function Design:
 - Poor hash functions lead to clustering and inefficiency.
- Dynamic Resizing:
 - Hash tables may need resizing to maintain performance.